

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH  
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



DSA-Extended

---

Báo cáo bài tập lớn (Mở rộng)  
**PII-NER-ANONYMIZER**

---

GVHD: Vương Bá Thịnh

|                      |                     |         |
|----------------------|---------------------|---------|
| Sinh viên thực hiện: | Trần Đình Ý         | 2414100 |
|                      | Trần Văn Minh Triết | 2413603 |
|                      | Nguyễn Phúc Vĩnh    | 2414001 |
|                      | Nguyễn Đình Minh    | 2412079 |

HỒ CHÍ MINH, THÁNG 5/ 2026

## **Tóm tắt nội dung**

Bài tập này tập trung vào việc

# Mục lục

|                                                       |          |
|-------------------------------------------------------|----------|
| Tóm tắt nội dung                                      | i        |
| Mục lục                                               | ii       |
| Danh sách hình vẽ                                     | iii      |
| Danh sách bảng                                        | iv       |
| Danh sách đoạn mã                                     | v        |
| <b>1 Giới thiệu</b>                                   | <b>1</b> |
| 1.1 Nhận diện tên riêng và địa chỉ (Context Detector) | 2        |
| 1.1.1 Nhận diện tên người                             | 2        |
| 1.1.2 Nhận diện địa chỉ                               | 7        |
| 1.2 Hợp nhất thực thể (Merger)                        | 8        |
| 1.2.1 Vấn đề overlap                                  | 9        |
| 1.2.2 Hệ thống ưu tiên                                | 9        |
| 1.2.3 Thuật toán Greedy Priority-First                | 9        |
| 1.2.4 Minh họa trường hợp overlap                     | 10       |

## Danh sách hình vẽ

## Danh sách bảng

|     |                                                           |    |
|-----|-----------------------------------------------------------|----|
| 1.1 | Bảng tín hiệu chấm điểm cho candidate tên người . . . . . | 4  |
| 1.2 | Danh sách thực thể trước khi hợp nhất . . . . .           | 10 |
| 1.3 | Danh sách thực thể sau khi hợp nhất . . . . .             | 11 |

## Danh sách đoạn mã

**PHÂN CÔNG NHIỆM VỤ VÀ KẾT QUẢ CỦA TỪNG  
THÀNH VIÊN**

| STT | HỌ VÀ TÊN              | MSSV    | NHIỆM VỤ | KẾT QUẢ |
|-----|------------------------|---------|----------|---------|
| 1   | Trần Văn Minh<br>Triết | 2413603 | None.    | 100%    |
| 2   | Nguyễn Đình Minh       | 2412079 | None.    | 100%    |
| 3   | Nguyễn Phúc Vĩnh       | 2414001 | None.    | 100%    |
| 4   | Trần Đình Ý            | 2414100 | None.    | 100%    |



# 1 Giới thiệu

Trong bài tập này



## 1.1 Nhận diện tên riêng và địa chỉ (Context Detector)

Module `context_detector` chịu trách nhiệm phát hiện ba loại thực thể dựa trên ngữ cảnh xung quanh: tên người (`NAME`), tên người dùng (`USERNAME`) và địa chỉ (`ADDRESS`). Khác với `regex_detector` vốn chỉ dựa vào cấu trúc cố định của chuỗi ký tự, module này phân tích ý nghĩa ngữ nghĩa của văn bản để xác định thực thể. Phần dưới đây tập trung trình bày chi tiết hai thành phần chính: nhận diện tên người và nhận diện địa chỉ.

### 1.1.1 Nhận diện tên người

Nhận diện tên người là bài toán khó nhất trong ba loại thực thể vì tên người không tuân theo một cấu trúc cố định và có thể xuất hiện trong rất nhiều ngữ cảnh khác nhau. Thuật toán được thiết kế theo hai tầng xử lý: *fast path* khi có dấu hiệu rõ ràng, và *scoring path* khi cần suy luận từ ngữ cảnh.

**Tiền xử lý — Tách câu thông minh.** Trước khi phân tích ngữ cảnh, văn bản đầu vào được tách thành các câu riêng lẻ. Đây là bước quan trọng vì điểm ngữ cảnh (pronoun, nghề nghiệp, trigger) chỉ có ý nghĩa trong phạm vi một câu.

Thách thức chính là phân biệt dấu chấm kết thúc câu với dấu chấm trong các trường hợp khác. Hàm `_split_sentences(text)` xử lý điều này theo quy tắc: dấu `.` chỉ được coi là ranh giới câu khi *đồng thời* thỏa ba điều kiện:

1. Token kết thúc bằng `.` không thuộc danh sách viết tắt đã biết (`Dr.`, `Mr.`, `St.`, `Apt.`, `Inc.`, v.v.).
2. Phần văn bản ngay sau `.` không bắt đầu bằng tên miền cấp cao (`com`, `edu`, `vn`, `org`, v.v.) — nhằm tránh split nhầm địa chỉ email và URL.
3. Sau dấu `.` là khoảng trắng hoặc hết chuỗi.

Dấu `!` và `?` luôn được coi là kết thúc câu vô điều kiện.

**Tầng 1 — Fast path (Trigger mạnh).** Nếu văn bản chứa một trong các cụm từ trigger, tên được xác định ngay lập tức mà không cần qua bước chấm điểm. Trigger được chia thành hai nhóm:

- **NAME\_TRIGGERS:** cụm từ chỉ thẳng tên chủ thể, gồm `"my name is"`, `"full name"`, `"name:"`.
- **\_LABEL\_TRIGGERS:** nhãn hay đứng trước tên trong văn bản có cấu trúc, gồm khoảng 25 mẫu như `"contact information:"`, `"author:"`, `"submitted by:"`, `"regards,"`, `"sincerely,"`, v.v.

Sau khi tìm thấy trigger, thuật toán thu thập 1–4 token liên tiếp thỏa: bắt đầu bằng chữ hoa, không chứa chữ số, không chứa ký tự đặc biệt (ngoại trừ dấu gạch ngang – vì tên có thể có dạng **Mary-Jane**), không phải stopword, và không mang đuôi động từ sau token đầu tiên (**-ing**, **-tion**, **-ed**, v.v.).

**Tầng 2 — Scoring path (Chấm điểm ngữ cảnh).** Khi không có trigger, thuật toán chuyển sang scoring path gồm ba bước.

**Bước 2.1 — Quét candidate.** Hàm `_extract_name_candidates(text)` duyệt toàn bộ văn bản, tìm các cụm từ viết hoa liên tiếp có độ dài từ 2 đến 4 token. Các điều kiện loại trừ được áp dụng tại hai mức:

- *Outer loop* (loại trước khi thu thập): token bắt đầu bằng chữ số, token trước là số nguyên thuần túy (dấu hiệu địa chỉ), là title prefix (**Dr.**, **Mr.**, v.v. — strip và tiếp tục), là **As** đứng đầu câu, là stopword, hoặc là từ tiêu đề section (**Information**, **Details**, **Witness**, v.v.).
- *Inner loop* (dừng thu thập giữa chừng): gặp token thường (không hoa), chứa ký tự đặc biệt, là chức danh công việc (**Manager**, **Director**, **Account**, v.v.), mang đuôi động từ sau token đầu tiên, hoặc bắt đầu bằng chữ số.

Sau khi thu thập xong, một candidate bị loại nếu token cuối cùng là hậu tố chỉ tổ chức hoặc địa điểm (**University**, **Company**, **Hospital**, **Hotel**, v.v.) — nhóm này gồm khoảng 70 từ chia thành bốn loại: công ty/doanh nghiệp, tổ chức/cơ quan, trường học, và địa điểm/cơ sở vật chất.

**Bước 2.2 — Chấm điểm.** Hàm `_score_candidate(candidate, sentence, full_text)` tính điểm cho từng cặp (candidate, câu). Một candidate xuất hiện qua nhiều câu sẽ lấy điểm cao nhất trong tất cả các câu đó. Bảng 1.1 tổng hợp các tín hiệu và điểm tương ứng.

Bảng 1.1: Bảng tín hiệu chấm điểm cho candidate tên người

| Tín hiệu                                    | Điểm       | Ghi chú                        |
|---------------------------------------------|------------|--------------------------------|
| Trigger mạnh + candidate đứng ngay sau      | 100, break | Dừng ngay, không so sánh       |
| I, [name] hoặc [name], I                    | 100, break |                                |
| I'm [name] hoặc I am [name]                 | 100, break |                                |
| I was named [name]                          | 100, break |                                |
| Từ my name xuất hiện trong câu              | +10        |                                |
| As [name], đứng đầu câu                     | +5         |                                |
| Dấu : đứng ngay trước candidate             | +5         | Bất kỳ label nào               |
| Appositive: [name], <chức danh> ,           | +3         | Từ sau phải là nghề nghiệp     |
| Chữ ký: [name] \n<chức danh>                | +4         | Candidate trên dòng riêng      |
| Đại từ ngôi 1 (I, me, my) trong câu         | +2         | Ưu tiên hơn ngôi 3             |
| Đại từ ngôi 3 (he, she, his, her)           | +1         | Chỉ tính khi không có ngôi 1   |
| Nghề nghiệp xuất hiện trong cùng câu        | +2         | Xem phần phát hiện nghề nghiệp |
| Tên lặp lại trong toàn đoạn (partial match) | +1/lần     | Tối đa +5                      |
| Title prefix (Dr., Mr., v.v.) trước tên     | +1         |                                |

Lưu ý rằng đại từ không cộng dồn nếu xuất hiện nhiều lần trong cùng một câu. Nếu câu đồng thời có đại từ ngôi 1 và ngôi 3 thì chỉ tính điểm cho ngôi 1.

**Phát hiện nghề nghiệp.** Hàm `_has_job_in_sentence(sentence)` trả về True nếu câu chứa tín hiệu nghề nghiệp theo một trong hai cách: (1) từ nằm trong danh sách cứng khoảng 35 nghề (doctor, lawyer, developer, CEO, v.v.) bất kể vị trí trong câu; (2) từ mang đuôi nghề nghiệp (-er, -ist, -or, -ician, v.v.) và đứng ngay sau một role marker như a, an, is a, works as. Điều kiện role marker giúp tránh false positive như underwater hay after.



**Bước 2.3 — Chọn winner.** Sau khi chấm điểm toàn bộ candidate, winner được chọn theo quy tắc: nếu có candidate đạt điểm 100 (definitive) thì trả về ngay; nếu chỉ có một candidate thì trả về candidate đó dù điểm bằng 0; nếu nhiều candidate thì chọn điểm cao nhất, tie-breaking theo vị trí xuất hiện sớm hơn trong văn bản.

---

**Algorithm 1** Thuật toán nhận diện tên người

---

**Require:** Văn bản đầu vào  $T$

**Ensure:** Danh sách thực thể tên  $E$

```
1:  $E \leftarrow \emptyset$ 
2:  $lower \leftarrow \text{lowercase}(T)$ 
// Tầng 1: Fast path
3: for mỗi trigger  $tr$  trong  $\text{NAMETRIGGERS} \cup \text{LABELTRIGGERS}$  do
4:   for mỗi vị trí  $pos$  của  $tr$  trong  $lower$  do
5:     Thu thập 1–4 token hoa liên tiếp sau  $pos$  vào  $tokens$ 
6:     if  $tokens \neq \emptyset$  then
7:       Thêm  $\text{join}(tokens)$  vào  $E$  với điểm 100
8:     end if
9:   end for
10: end for
11: if  $E \neq \emptyset$  then return  $E$ 
12: end if
// Tầng 2: Scoring path
13:  $sentences \leftarrow \text{SPLITSENTENCES}(T)$ 
14:  $candidates \leftarrow \text{EXTRACTCANDIDATES}(T)$ 
15: if  $candidates = \emptyset$  then return  $\emptyset$ 
16: end if
17:  $bestScores \leftarrow \{\}$  //  $candidate \rightarrow (score, start, end)$ 
18: for mỗi candidate  $c$  trong  $candidates$  do
19:    $best \leftarrow 0$ 
20:   for mỗi câu  $s$  trong  $sentences$  do
21:     if không có phần nào của  $c$  trong  $s$  then continue
22:   end if
23:    $(score, def) \leftarrow \text{SCORECANDIDATE}(c, s, T)$ 
24:   if  $def = \text{True}$  then
25:     return  $\{c\}$  // Definitive, dừng ngay
26:   end if
27:    $best \leftarrow \max(best, score)$ 
28: end for
29: Cập nhật  $bestScores[c]$  nếu  $best$  cao hơn giá trị hiện tại
30: end for
31:  $winner \leftarrow \arg \max_c bestScores[c]$  // Tie: chọn cái xuất hiện sớm hơn
32: return  $\{winner\}$ 
```

---

### 1.1.2 Nhận diện địa chỉ

Địa chỉ trong tập dữ liệu tuân theo cấu trúc điển hình của địa chỉ Mỹ gồm số nhà, tên đường, loại đường, và hậu tố hướng hoặc đơn vị căn hộ tùy chọn. Module nhận diện địa chỉ sử dụng hai cơ chế bổ sung cho nhau.

**Cơ chế 1 — Regex pattern.** Biểu thức chính quy `_ADDRESS_RE` mô tả cấu trúc địa chỉ theo bốn thành phần nối tiếp:

1. **Số nhà:** `\b(\d{1,5})(?!\d)\s+` — từ 1 đến 5 chữ số, kèm negative lookahead `(?!\d)` để đảm bảo đây không phải là một phần của số dài hơn (loại trừ số điện thoại, mã bưu điện dài).
2. **Tên đường:** `((?:[A-Z0-9][A-Za-z0-9]*\s+)+?)` — một hoặc nhiều từ bắt đầu bằng chữ hoa hoặc chữ số, sử dụng non-greedy để nhường ưu tiên cho thành phần loại đường phía sau.
3. **Loại đường:** danh sách khoảng 30 từ được sắp xếp từ dài đến ngắn để tránh match sớm, bao gồm dạng đầy đủ (Boulevard, Terrace, Avenue, Street, Drive, Road, v.v.) và dạng viết tắt (Blvd., Ave., Rd., St., v.v.).
4. **Hướng và đơn vị** (tùy chọn): hướng gồm tám giá trị North, South, East, West và các tổ hợp; đơn vị gồm Suite, Apt., Apartment, Unit kèm số phòng.

Trước khi kiểm tra kết quả regex, tất cả span email trong văn bản được xác định trước; bất kỳ span địa chỉ nào nằm hoàn toàn bên trong span email sẽ bị bỏ qua để tránh nhầm lẫn phần tên miền của địa chỉ email với địa chỉ nhà.

**Cơ chế 2 — Trigger keyword (fallback).** Đối với địa chỉ không có loại đường tiêu chuẩn, module sử dụng danh sách trigger bao gồm `street`, `road`, `avenue`, `district`, `city`, `st.`, `rd.`, `ave.`. Khi tìm thấy trigger, thuật toán lấy clause xung quanh (từ dấu câu trước đến dấu câu sau), loại bỏ noise words đầu clause (`at`, `is`, `the`, `located`, `live`, `reside`, v.v.), và kiểm tra thêm: số nhà ở đầu clause không được vượt quá 5 chữ số, và span không được trùng lặp với kết quả từ cơ chế regex.

---

**Algorithm 2** Thuật toán nhận diện địa chỉ

---

**Require:** Văn bản đầu vào  $T$

**Ensure:** Danh sách thực thể địa chỉ  $E$

```
1:  $E \leftarrow \emptyset$ 
2:  $emailSpans \leftarrow \text{FINDEMAILSPANS}(T)$ 
// Cơ chế 1: Regex pattern
3: for mỗi match  $m$  của ADDRESSRE trong  $T$  do
4:   if  $m$  nằm trong một email span then continue
5:   end if
6:   Thêm span của  $m$  vào  $E$ 
7: end for
// Cơ chế 2: Trigger keyword fallback
8: for mỗi trigger  $tr$  trong ADDRESSTRIGGERS do
9:   for mỗi vị trí  $pos$  của  $tr$  trong  $T$  do
10:     $clause \leftarrow \text{EXTRACTCLAUSE}(T, pos)$ 
11:     $addr \leftarrow \text{STRIPLEADINGNOISE}(clause)$ 
12:    if số nhà đầu  $addr$  có nhiều hơn 5 chữ số then continue
13:    end if
14:    if  $addr$  nằm trong email span then continue
15:    end if
16:    if  $addr$  overlap với span đã có trong  $E$  then continue
17:    end if
18:    Thêm span của  $addr$  vào  $E$ 
19:   end for
20: end for
21: return  $\text{DEDUPLICATE}(E)$ 
```

---

## 1.2 Hợp nhất thực thể (Merger)

Sau khi `regex_detector` và `context_detector` chạy độc lập, mỗi module trả về một danh sách thực thể riêng. Hai danh sách này được đưa vào module `merger` để hợp nhất thành một danh sách duy nhất, sạch và không chồng lấn.

### 1.2.1 Vấn đề overlap

Hai thực thể được coi là *chồng lấn* (overlap) nếu khoảng ký tự của chúng giao nhau, tức là  $[s_1, e_1)$  và  $[s_2, e_2)$  thỏa  $s_1 < e_2$  và  $e_1 > s_2$ . Overlap xảy ra trong nhiều tình huống thực tế:

- Địa chỉ email `kazuosun@hotmail.net` bị `regex_detector` bắt đúng là **EMAIL**, nhưng `context_detector` có thể nhận nhầm phần `hotmail.net` là URL hoặc nhận phần trước `@` là **NAME**.
- Số điện thoại `+27 68 670 7513` có thể bị nhận vừa là **PHONE** vừa overlap với một span **ADDRESS** nếu đứng sau một địa chỉ.
- Tên người `Baha Hoffman` xuất hiện ngay trước số điện thoại, `context_detector` có thể kéo dài span sang phần số.

### 1.2.2 Hệ thống ưu tiên

Để giải quyết overlap một cách nhất quán, merger định nghĩa một thứ tự ưu tiên tuyến tính cho sáu loại thực thể:

**EMAIL > URL > PHONE > ADDRESS > NAME > USERNAME**

Thứ tự này phản ánh độ tin cậy của từng detector: **EMAIL**, **URL** và **PHONE** được phát hiện bằng regex có độ chính xác cao và cấu trúc rõ ràng nên được ưu tiên hơn; **ADDRESS**, **NAME**, **USERNAME** dựa trên ngữ cảnh nên được xếp sau.

### 1.2.3 Thuật toán Greedy Priority-First

Hàm `resolve_overlaps` thực hiện giải quyết overlap theo chiến lược tham lam ưu tiên (greedy priority-first):

1. Gộp tất cả thực thể từ mọi detector thành một danh sách duy nhất.
2. Sắp xếp danh sách theo ba tiêu chí lần lượt: (a) thứ hạng ưu tiên tăng dần (loại ưu tiên cao xếp trước), (b) chỉ số **start** tăng dần, (c) độ dài span giảm dần (span dài hơn xếp trước khi cùng điểm xuất phát).
3. Duyệt tuần tự qua danh sách đã sắp xếp. Với mỗi thực thể, giữ lại nếu span của nó không chồng lấn với bất kỳ thực thể đã giữ nào; bỏ qua nếu ngược lại.
4. Sắp xếp lại danh sách kết quả theo **start** để đảm bảo thứ tự xuất hiện trong văn bản.



---

**Algorithm 3** Thuật toán hợp nhất và giải quyết overlap (Greedy Priority-First)

---

**Require:** Các nhóm thực thể  $G_1, G_2, \dots, G_k$  từ các detector

**Ensure:** Danh sách thực thể không overlap, sắp theo vị trí

```
1:  $all \leftarrow G_1 \cup G_2 \cup \dots \cup G_k$ 
2: Sắp xếp  $all$  theo  $(priority(type), start, -(end - start))$  tăng dần
3:  $kept \leftarrow \emptyset$ 
4:  $occupied \leftarrow \emptyset$  // Tập các span  $(s, e)$  đã giữ
5: for mỗi thực thể  $ent$  trong  $all$  do
6:    $s \leftarrow ent.start, e \leftarrow ent.end$ 
7:   if  $\nexists (os, oe) \in occupied$  sao cho  $s < oe$  và  $e > os$  then
8:      $kept \leftarrow kept \cup \{ent\}$ 
9:      $occupied \leftarrow occupied \cup \{(s, e)\}$ 
10:  end if
11: end for
12: Sắp xếp  $kept$  theo  $start$  tăng dần
13: return  $kept$ 
```

---

#### 1.2.4 Minh họa trường hợp overlap

Xét đoạn văn bản sau được trích từ tập dữ liệu:

*“please do not hesitate to contact Baha Hoffman at +27 68 670 7513 or via email at  
bahahoffman@yahoo.net.”*

Hai detector tạo ra các thực thể như trong Bảng 1.2.

Bảng 1.2: Danh sách thực thể trước khi hợp nhất

| Detector         | Loại  | Text                  | Ưu tiên |
|------------------|-------|-----------------------|---------|
| regex_detector   | PHONE | +27 68 670 7513       | 3       |
| regex_detector   | EMAIL | bahahoffman@yahoo.net | 1       |
| context_detector | NAME  | Baha Hoffman          | 5       |
| context_detector | NAME  | Baha Hoffman at       | 5       |

Sau khi sắp xếp theo ưu tiên và áp dụng thuật toán greedy, merger loại bỏ Baha

Hoffman at vì overlap với Baha Hoffman đã được giữ trước đó (cùng loại NAME nhưng span ngắn hơn xếp trước). Kết quả cuối cùng được trình bày trong Bảng 1.3.

Bảng 1.3: Danh sách thực thể sau khi hợp nhất

| Loại  | Text                  | Vị trí            |
|-------|-----------------------|-------------------|
| NAME  | Baha Hoffman          | start=35, end=47  |
| PHONE | +27 68 670 7513       | start=51, end=66  |
| EMAIL | bahahoffman@yahoo.net | start=79, end=103 |

Kết quả này sau đó được đưa vào module `anonymizer` để thay thế các span PII bằng placeholder tương ứng, tạo ra văn bản đã được ẩn danh hóa.